

Doctor Appointment System

Technical Report & Implementation Guide

Doctor Appointment System

A Secure Web Application for Medical Booking

Generated: June 02, 2026

Platform: Render Cloud (PostgreSQL + Gunicorn)

Framework: Flask 3.x (Python)

Table of Contents

1. Introduction & Objectives
2. System Architecture
3. Database Design & Schema
4. Core Features & Implementation
 - 4.1 User Authentication (Registration & Login)
 - 4.2 Patient Dashboard & Doctor Search
 - 4.3 Appointment Booking with Document Upload
 - 4.4 Doctor Schedule Management
 - 4.5 Appointment Cancellation
5. Security Architecture
 - 5.1 Password Security & Strength Enforcement
 - 5.2 CSRF Protection
 - 5.3 Input Sanitization (XSS Prevention)
 - 5.4 Rate Limiting & Brute-Force Protection
 - 5.5 Session Management & Inactivity Timeout
 - 5.6 JWT-Based API Authentication
 - 5.7 File Upload Security (Magic Byte Validation)
 - 5.8 Access Control & Authorization
6. REST API Design
7. Frontend Design & UX
8. Deployment on Render

Doctor Appointment System

Technical Report & Implementation Guide

9. Technologies & Dependencies

10. Conclusion

Doctor Appointment System

Technical Report & Implementation Guide

1. Introduction & Objectives

The Doctor Appointment System is a full-stack web application designed to streamline the process of booking medical consultations. It provides two distinct user portals: a Patient Portal for searching doctors, booking appointments, and uploading medical documents, and a Doctor Portal for viewing daily schedules and accessing patient-submitted clinical documents.

The system was built with security as a foundational principle. Every endpoint enforces role-based access control, all user inputs are sanitized against injection attacks, file uploads are validated at the binary level, and session tokens are protected with best-practice cookie flags. The application was deployed on Render cloud infrastructure using PostgreSQL and Gunicorn for production-grade reliability.

Key objectives achieved:

- Dual-role portal with distinct patient and doctor workflows
- Secure appointment booking with real-time slot availability
- Medical document upload with validation at the file signature (magic byte) level
- RESTful JSON API for external client integration
- Comprehensive security: CSRF, rate limiting, XSS sanitization, JWT, session timeout
- Cloud deployment with automatic horizontal scaling and managed PostgreSQL

2. System Architecture

The application follows a Model-View-Controller (MVC) pattern implemented with Flask. The architecture is organized into four primary layers:

- Presentation Layer (Templates): Jinja2 templates with a glassmorphism dark theme providing responsive HTML5 interfaces for login, registration, dashboard, booking, and doctor schedule views.
- Business Logic Layer (app.py): Central Flask application handling routing, form validation, session management, and access control enforcement for 15 routes.
- Data Access Layer (database.py): Abstracted database interface with DbConnection and DbCursor wrappers that transparently convert SQLite parameter syntax to PostgreSQL, providing seamless dual-database support.
- Security Layer (security.py): Cross-cutting security module providing password hashing, input sanitization (Bleach), JWT token generation/validation, rate limiting, and file upload validation.

The application supports two database backends: SQLite for local development and PostgreSQL for production on Render. The DbCursor wrapper automatically converts "?" placeholders to "%s" when PostgreSQL is detected via the DATABASE_URL environment variable, allowing the same codebase to run in both environments without modification.

3. Database Design & Schema

The database consists of three relational tables with foreign key constraints and unique indexes ensuring data integrity:

3.1 Users Table

id	INTEGER PRIMARY KEY AUTOINCREMENT (SQLite) SERIAL PRIMARY KEY (PostgreSQL)
name	TEXT NOT NULL

Doctor Appointment System

Technical Report & Implementation Guide

email	TEXT UNIQUE NOT NULL
phone	TEXT NOT NULL
password_hash	TEXT NOT NULL -- bcrypt hashed (12 rounds)
role	TEXT NOT NULL CHECK(role IN (patient, doctor))
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP

The users table stores both patient and doctor accounts. The role column enforces a strict check constraint limiting values to "patient" or "doctor". Passwords are never stored in plaintext; they are hashed using bcrypt with a work factor of 12 rounds before storage.

3.2 Appointments Table

id	INTEGER PRIMARY KEY AUTOINCREMENT
patient_id	INTEGER NOT NULL -- FK -> users(id)
doctor_id	INTEGER NOT NULL -- FK -> users(id)
date	TEXT NOT NULL -- YYYY-MM-DD format
time_slot	TEXT NOT NULL -- e.g. "10:00 AM"
reason	TEXT NOT NULL -- Sanitized visit reason
document_path	TEXT -- Optional uploaded file
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP
UNIQUE(doctor_id, date, time_slot) -- Prevent double-booking	

The appointments table enforces a unique constraint on (doctor_id, date, time_slot) to prevent race conditions where two patients could book the same slot simultaneously. The database-level constraint serves as the final safety net after application-level validation.

3.3 Login Attempts Table

id	INTEGER PRIMARY KEY AUTOINCREMENT
email	TEXT NOT NULL
ip_address	TEXT NOT NULL
timestamp	DATETIME DEFAULT CURRENT_TIMESTAMP
success	INTEGER NOT NULL CHECK(success IN (0, 1))

This table powers the rate limiting system. Failed login attempts are logged with their IP address and timestamp. The check_rate_limit function queries this table to determine if an IP has exceeded the threshold (5 failures in 15 minutes).

Doctor Appointment System

Technical Report & Implementation Guide

4. Core Features & Implementation

4.1 User Authentication (Registration & Login)

Registration (POST /register): New patients complete a multi-field form (name, email, phone, password). The form enforces client-side validation via JavaScript (real-time password strength indicator with 5 criteria: length ≥ 8 , lowercase, uppercase, digit, special character) and server-side validation in Python using regex patterns. Passwords are hashed with bcrypt (gensalt with 12 rounds) before storage. Duplicate email detection prevents multiple accounts.

Login (POST /login): The login page features a role tab-switcher (Patient/Doctor). Credentials are validated against the database using bcrypt.checkpw(). On success, the session is cleared (preventing session fixation attacks), then populated with user_id, name, and role. A last_activity timestamp is set for inactivity tracking. Failed attempts are logged to the login_attempts table for rate limiting.

As a practical example, a patient user "John Smith" would register with email "john@example.com" and password "StrongP@ss1", then log in to access the patient dashboard. A doctor such as "Dr. Alice Smith" logs in with "alice.smith@hospital.com" and the default seeded password "SecurePassword123!" to view their appointment schedule.

4.2 Patient Dashboard & Doctor Search

The dashboard (GET /dashboard) serves as the patient's command center. It displays all scheduled appointments in a tabular format showing the doctor's name, date, time slot, reason for visit, uploaded documents, and status badge. An empty state with a call-to-action button is shown when no appointments exist.

The integrated doctor search uses a GET form with query parameter "search" that performs a SQL LIKE query against doctor names and email addresses. Results are displayed with a "Book Slot" button that links to the booking page with the doctor pre-selected. Search inputs are sanitized through the security.sanitize_text() function to prevent injection.

4.3 Appointment Booking with Document Upload

The booking page (GET/POST /book) implements a multi-step form:

- Doctor Selection: Dropdown populated from the users table (role = doctor)
- Date Picker: HTML5 date input restricted to weekdays (Mon-Fri) and future dates
- Time Slot Grid: Dynamic 12-slot grid (30-min intervals, 9 AM - 12 PM and 2 PM - 4:30 PM). Booked slots are fetched via a REST API call to /api/appointments and displayed as disabled with strikethrough styling.
- Reason Textarea: 500-character limit with real-time counter
- Document Upload: Drag-and-drop zone accepting PDF, JPG, JPEG, PNG (max 2MB). Client-side extension and size checks precede server-side validation.

The server-side booking logic performs: date validation (no past dates, weekdays only), time slot validation against a whitelist, file upload validation (extension, size, magic byte signature), and a double-booking check query against the database UNIQUE constraint.

4.4 Doctor Schedule Management

The doctor schedule view (GET /doctor/schedule) presents a daily agenda with a date filter that auto-submits on change. The table displays: time slot (accent-colored), patient name, contact phone, reason for visit (styled with a left accent border), and medical document download links. An empty state with a calendar icon placeholder appears when no consultations are scheduled for the selected date.

Doctor Appointment System

Technical Report & Implementation Guide

4.5 Appointment Cancellation

Patients can cancel appointments via POST `/cancel/<appointment_id>`. The route validates that the current user owns the appointment before executing the DELETE. A JavaScript confirmation dialog prevents accidental cancellations. The corresponding REST API endpoint (DELETE `/api/appointments/<id>`) provides the same functionality for programmatic clients.

Doctor Appointment System

Technical Report & Implementation Guide

5. Security Architecture

5.1 Password Security & Strength Enforcement

- bcrypt hashing with 12 salt rounds (industry standard)
- 5-criteria strength validation: 8+ chars, upper, lower, digit, special character
- Real-time visual strength meter with color-coded bar (red/amber/green)
- Checklist UI showing which criteria are met in real-time

5.2 CSRF Protection

Flask-WTF CSRFProtect is enabled globally. Every form includes a hidden `csrf_token` field generated by the server. API endpoints are exempted from CSRF because they use token-based authentication (JWT Bearer tokens). A custom CSRF error handler catches validation failures and redirects users with a friendly message.

5.3 Input Sanitization (XSS Prevention)

All user-supplied text (search queries, appointment reasons) is processed through `security.sanitize_text()`, which uses the Bleach library to strip all HTML tags. This prevents stored and reflected XSS attacks. The sanitization function allows zero HTML tags and zero attributes, effectively rendering any injected markup as plain text.

5.4 Rate Limiting & Brute-Force Protection

The `check_rate_limit` function queries the `login_attempts` table to count failed attempts from a given IP address within a 15-minute window. If the count exceeds 5 failures, the login is blocked with a user-facing message. The function uses Python-computed timestamps (database-agnostic) rather than database-specific date functions, ensuring compatibility across SQLite and PostgreSQL.

5.5 Session Management & Inactivity Timeout

- Session permanent lifetime set to 15 minutes
- Every request updates a `last_activity` timestamp in the session
- Stale sessions (older than 15 min) trigger automatic logout with redirect to login
- Client-side inactivity watchdog: JavaScript timer warns at 14 minutes, forces logout at 15 minutes with a fixed-position warning banner
- Session fixation prevention: `session.clear()` before populating new session on login
- Secure cookie flags: `HttpOnly=True`, `SameSite=Lax`, `Secure=True` in production

5.6 JWT-Based API Authentication

The REST API supports dual authentication: JWT Bearer tokens (preferred) and session cookies (fallback). The `get_authenticated_user()` function checks the Authorization header for a Bearer token, decodes it using PyJWT with HS256 algorithm, and extracts the user ID and role. Tokens expire after 2 hours. Invalid or expired tokens return 401 Unauthorized. The JWT secret is generated using Python's `secrets.token_hex(32)` if not set via environment variable, ensuring each deployment gets a unique, cryptographically strong key.

5.7 File Upload Security (Magic Byte Validation)

File uploads undergo a comprehensive 3-layer validation:

- Extension check: Only .pdf, .jpg, .jpeg, .png are allowed

Doctor Appointment System

Technical Report & Implementation Guide

- Size check: Files are limited to 2MB (MAX_CONTENT_LENGTH)
- Magic byte signature check: The first 16 bytes of each file are read and compared against known file signatures (%PDF, \xFF\xD8\xFF for JPEG, \x89PNG for PNG). This prevents MIME type spoofing attacks.

Files are saved with a timestamp-prefixed secure filename (via Werkzeug's secure_filename) to the uploads/ directory, preventing path traversal attacks.

5.8 Access Control & Authorization

- Role-based route protection: Patient routes check session["role"] == "patient", doctor routes check session["role"] == "doctor"
- Document access control: The view_document endpoint verifies the requesting user is either the patient who uploaded the document or the assigned doctor
- API endpoints validate JWT/subject against requested resources
- Cancellation endpoint verifies appointment ownership before executing DELETE

Doctor Appointment System

Technical Report & Implementation Guide

6. REST API Design

The application exposes 5 JSON API endpoints for programmatic access:

POST /api/token - Obtain JWT Token

```
Request: {"email": "...", "password": "...", "role": "patient"}
Response: {"token": "<jwt>", "expires_in_hours": 2}
```

Authenticates a user and returns a signed JWT token valid for 2 hours.

GET /api/appointments - List Appointments

```
Params: doctor_id, date (optional - checks slot availability)
Response: {"appointments": [{"time_slot": "10:00 AM"}, ...]}
```

When called with doctor_id and date parameters, returns booked time slots for availability checking. When called with valid authentication, returns the authenticated patient's bookings.

POST /api/appointments/book - Book Appointment

```
Request: {"doctor_id": 1, "date": "2026-06-15",
          "time_slot": "10:00 AM", "reason": "Checkup"}
Response: {"success": true, "message": "Appointment booked successfully"}
```

GET /api/doctor/schedule - Doctor Schedule

```
Params: date (YYYY-MM-DD)
Response: {"date": "2026-06-15",
          "schedule": [{"id": 1, "time_slot": "09:00 AM",
                        "patient": "John Doe", ...}]}
```

Returns a doctor's scheduled consultations for a specific date.

DELETE /api/appointments/<id> - Cancel Appointment

```
Response: {"success": true, "message": "Appointment cancelled"}
```

Cancels an appointment. Requires patient authentication and appointment ownership.

7. Frontend Design & UX

The frontend is built with semantic HTML5, custom CSS3, and vanilla JavaScript. No external CSS or JS frameworks are used, minimizing dependency bloat and potential supply chain risks.

- Dark theme with CSS custom properties (--bg-main: #0a0d1a) and gradient backgrounds
- Glassmorphism card design using backdrop-filter: blur(16px) for a modern aesthetic
- Outfit/Inter font family via Google Fonts for clean, professional typography
- Responsive grid layout: 2-column dashboard (2fr + 1fr) collapsing to single column on mobile (768px breakpoint)
- Interactive elements: password strength meter, tab switcher for login roles, drag-and-drop file upload zone, AJAX slot availability fetching
- Inline SVG icons throughout for visual affordance (calendar, search, upload, user icons)
- Status badges (scheduled/completed) with semi-transparent colored backgrounds

Doctor Appointment System

Technical Report & Implementation Guide

- Gradient buttons with hover animations (translateY + box-shadow transitions)

8. Deployment on Render

The application is deployed on Render.com using the Blueprint infrastructure-as-code approach. The render.yaml file at the repository root defines the full deployment specification:

- Web Service: Python 3 environment, Oregon region, Free tier plan
- Build Command: pip install -r requirements.txt
- Start Command: gunicorn app:app (production WSGI server)
- Auto-scaling: Gunicorn worker processes managed by Render
- Environment Variables: SECRET_KEY and JWT_SECRET auto-generated with secure random values via generateValue: true in render.yaml
- PostgreSQL: Automatically provisioned with DATABASE_URL injected into the environment
- Auto-deploy: Connected to GitHub repository; Render automatically redeploys on every git push to the main branch

The database module detects the DATABASE_URL environment variable and automatically switches from SQLite to PostgreSQL, applying the necessary placeholder conversions transparently. Session cookies are automatically configured as Secure=True in production (detected by FLASK_DEBUG=0), ensuring compliance with HTTPS requirements.

9. Technologies & Dependencies

Core Framework & Libraries:

- Flask 3.x - Python web micro-framework for routing, templating, and session management
- Flask-WTF - CSRF protection integration with form validation
- Jinja2 (bundled with Flask) - Server-side HTML template rendering
- Bleach 6.x - HTML sanitization library for XSS prevention
- bcrypt 5.x - Password hashing with 12-round salt generation
- PyJWT 2.x - JSON Web Token encoding and decoding (HS256)
- python-dotenv - Environment variable loading from .env files
- psycopg2-binary - PostgreSQL adapter for Python
- Gunicorn - Production-grade WSGI HTTP server

Database:

- SQLite 3 (local development) - File-based relational database
- PostgreSQL (production on Render) - Managed cloud database

Frontend:

- HTML5, CSS3, vanilla JavaScript (no frameworks)
- Google Fonts: Inter (body), Outfit (headings)

10. Conclusion

Doctor Appointment System

Technical Report & Implementation Guide

The Doctor Appointment System successfully delivers a secure, production-ready medical booking platform with the following key achievements:

- Complete dual-role workflow: Patients browse doctors, book appointments, upload documents, and cancel bookings. Doctors view daily schedules with patient details and clinical documents.
- Defense-in-depth security: Multiple overlapping security layers including bcrypt hashing, CSRF tokens, XSS sanitization, rate limiting, JWT authentication, magic byte file validation, session timeout, and strict role-based access control.
- Database abstraction layer: The DbConnection/DbCursor wrappers provide transparent SQLite-to-PostgreSQL migration, enabling seamless local development and cloud deployment from a single codebase.
- REST API: 5 JSON endpoints with JWT Bearer authentication enable external client integration (mobile apps, third-party systems).
- Cloud-ready deployment: render.yaml Blueprint spec enables one-click deployment on Render with auto-provisioned PostgreSQL, auto-generated secrets, and continuous deployment from GitHub.

All 5 security unit tests pass successfully (password strength, XSS sanitization, rate limiting, JWT token generation/validation, and API endpoint protection). The codebase follows industry best practices for web security, database design, and cloud deployment.

--- End of Report ---

Doctor Appointment System - Technical Report

Generated on June 02, 2026 at 12:53